

HBM4 Signal Flow — Block by Block Explanation

BLOCK 1: The Processor Side (AXI Interface)

Think of the processor as someone placing an order at a counter. It doesn't care about what happens behind the counter — it just wants to write data somewhere or read data back.

How a WRITE starts:

The processor first raises AWVALID to say *"I have an address I want to write to."* Along with that, it puts the destination address on AWADDR, tells the DUT how many beats of data are coming via AWLEN, how big each beat is via AWSIZE, and what pattern the addresses follow via AWBURST (incremental, fixed, or wrap-around). It also tags the transaction with a unique ID on AWID so it can track which response belongs to which request later.

Now the DUT needs to say it's ready. When the DUT asserts AWREADY, that single clock edge where both AWVALID and AWREADY are HIGH simultaneously — that is the handshake. The address is accepted. Neither side can back out of that moment.

After the address is accepted, the processor starts sending the actual data. It puts the first chunk of data on WDATA (128 bits wide — that's 16 bytes per beat), usesWSTRB to say which of those 16 bytes are actually valid (like a byte-enable mask), and raises WVALID. The DUT responds with WREADY when it can absorb the data. This repeats beat by beat. On the very last beat, the processor raises WLAST — this tells the DUT *"this is the final piece of data for this burst, don't wait for more."*

Once all data is received, the DUT sends back a response. It raises BVALID, puts the original AWID value back on BID so the processor knows which transaction completed, and drives BRESP — 00 means everything went fine (OKAY), 10 means something went wrong on the slave side (SLVERR). The processor confirms it received the response by asserting BREADY. That handshake closes the entire write transaction.

How a READ works:

It follows the same idea but simpler — there's no separate data phase going in. The processor raises ARVALID with the read address on ARADDR, burst info on ARLEN/ARSIZE/ARBURST, and a transaction ID on ARID. The DUT responds with ARREADY to accept it.

Then the DUT starts pushing data back. It drives RDATA with the memory contents, raises RVALID, and echoes back RID to match the original request. The processor pulls each beat by keeping RREADY high. When the last beat comes out, the DUT raises RLAST. That ends the read.

BLOCK 2: Inside the HBM Memory Controller

This block is the translator and the brain. It receives AXI transactions and converts them into the very specific command language that DRAM understands.

When an AXI write arrives:

The controller latches the AWADDR and breaks it apart. The address bits are not arbitrary — specific bits map to the row address, specific bits map to the column address, specific bits select

the bank, and specific bits select the bank group. This address decode step is critical because DRAM is not byte-addressable like a cache — you access it in rows and columns.

Once decoded, the controller checks its internal BANK_OPEN[15:0] register — a bitmask where each bit represents whether a particular bank already has an open row. If the target bank is already open with the correct row, it can skip straight to sending a WRITE command. If the bank is closed, it must first issue an ACTIVATE. If the bank is open but with a *different* row, it must first PRECHARGE to close it, then ACTIVATE the new row.

This decision-making is the core of the CMD FSM described in the next section.

The actual command goes out on CMD[2:0] — a 3-bit code. 001 means ACTIVATE, 011 means WRITE, 010 means READ, 100 means PRECHARGE, 101 means REFRESH. Alongside the command, BANK[3:0] and BANK_GROUP[1:0] tell the DRAM which bank group and bank to target. ROW_ADDR[14:0] carries the row for ACTIVATE, and COL_ADDR[9:0] carries the column for READ/WRITE.

The refresh timer:

Running silently in the background, there's a counter ticking toward tREFI — about 3.9 microseconds. When it expires, the controller raises REFRESH_REQ. If a transaction is in progress, the controller finishes it, then issues a PRECHARGE to close all open banks, then sends the REFRESH command. The PHY acknowledges with REFRESH_ACK. During this entire window, no new transactions can proceed. This is a real source of latency jitter in HBM systems.

ECC happens here too:

When ECC_EN is high, the controller computes a parity/syndrome value before sending write data out. On reads, it checks the incoming data against the stored syndrome. A single-bit flip can be corrected silently — ECC_ERR_CORR pulses to log it. A multi-bit error cannot be fixed — ECC_ERR_UNCORR goes high and the AXI response becomes SLVERR.

BLOCK 3: The PHY Layer

The PHY is the electrical translator. The controller speaks in clean digital commands — the PHY converts those into the precise differential signals, timing-controlled strobes, and calibrated voltages that the physical DRAM dies need.

Clock delivery:

The PHY drives CK_t and CK_c — a differential clock pair — continuously to the DRAM dies. The DRAM uses this clock to latch everything. CKE (Clock Enable) must be held HIGH for the DRAM to operate. If CKE goes LOW, the DRAM enters a power-down state and ignores all commands.

Command transmission:

The 7-bit CA[6:0] bus carries the encoded command and address information to the DRAM. It's a multiplexed bus — meaning the same wires carry the command on one clock edge and part of the address on the next edge. CS_n[3:0] selects which pseudo-channel or die stack is being targeted. When CS_n is LOW for a particular channel, that channel is listening to whatever is on CA.

Data flow:

DQ[127:0] is the bidirectional data highway. During a WRITE command, the PHY drives data onto DQ from the controller's write buffer. During a READ command, the DRAM drives data back onto DQ and the PHY receives it. The exact moment to sample DQ is determined by DQS_t and DQS_c — a differential data strobe that toggles in sync with the data. The PHY uses the DQS edges to know precisely when to latch DQ, which compensates for any propagation delay on the physical wires.

Initialization sequence:

When power comes up, RESET_n is held LOW by the PHY. This holds the DRAM in reset. After a required delay, RESET_n goes HIGH, and the PHY goes through a calibration and training sequence — aligning DQS timing, setting termination values, writing mode registers via MRS commands. Only when all of this completes does PHY_INIT_DONE go HIGH, signaling the controller that it's safe to start sending real commands.

BLOCK 4: DRAM Dies + Logic Die

This is where data actually lives. From a verification perspective, you model this as a behavioral memory — an associative array in SystemVerilog.

What happens on ACTIVATE:

When the DRAM receives CMD=001 with a bank and row address, it physically opens that row — pulling all the bit cells in that row onto the sense amplifiers. This takes tRCD nanoseconds. During this time, the DRAM is busy and cannot accept a READ or WRITE to that bank. Your controller must wait by counting clock cycles.

What happens on READ/WRITE:

After tRCD has elapsed, a READ or WRITE command arrives with a column address. On WRITE, the data arrives on DQ a few cycles after the command (write latency), and the DRAM stores it at mem[row][col]. On READ, the DRAM drives DQ after a delay called tCL (CAS latency) — typically 14–16 clock cycles. Your controller must count these cycles before expecting valid data on DQ.

What happens on PRECHARGE:

The PRECHARGE command closes the currently open row in a bank. The sense amplifiers are reset, and the row is written back. This takes tRP nanoseconds. Until tRP expires, the bank cannot be activated again. Your BANK_OPEN bitmask bit for that bank gets cleared.

What happens on REFRESH:

All banks must be precharged before refresh. The REFRESH command triggers internal DRAM circuitry to recharge every capacitor cell — because DRAM is volatile and cells lose charge over time. The entire DRAM is unavailable for tRFC (typically 350–550 ns) during this. After tRFC, everything is back to normal and the refresh counter resets.

Inter-Block Communication Summary

Processor → Controller: Pure AXI4 handshakes. The processor drives VALID signals; the controller drives READY signals. Nothing moves until both are HIGH simultaneously.

Controller → PHY: The controller drops a command on CMD[2:0] with address signals, and asserts CMD_VALID. The PHY encodes this onto the CA bus and toggles CS_n for the right channel on the exact clock edge.

PHY ↔ DRAM: Differential signaling over CK, CA, CS_n, DQ, and DQS. The DRAM is a passive responder — it reacts to what it sees on CA/CS_n and either accepts data from DQ or drives data back onto DQ.

DRAM → Controller (read data path): Data comes back on DQ after tCL cycles. The PHY captures it using DQS strobes, buffers it, and drives RD_DATA_VALID HIGH to tell the controller *"the data on DQ right now is valid, latch it."* The controller then forwards this data up through the AXI RDATA channel back to the processor.

The key insight for your UVM testbench: Your AXI agent only ever sees Block 1 signals. Your HBM monitor only sees Block 2/3 signals. The scoreboard sits above both — it watches what went in on AXI and what commands came out on the HBM side, and independently predicts what the answer should be using the reference model. If they match, the test passes.